

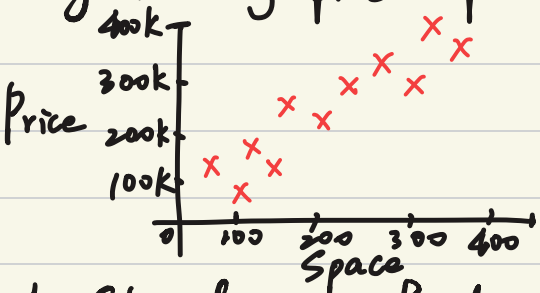
Made with Goodnotes

Machine Learning

1. Supervised Learning (监督学习) $X \rightarrow Y$ Learns from being given "Right Answers"

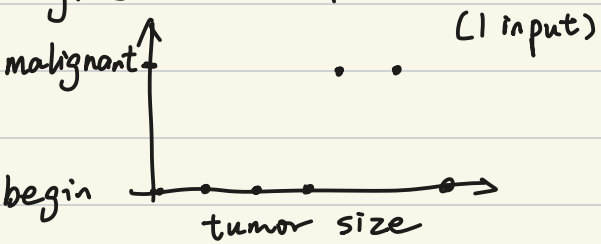
↳ Regression: Predict a number infinitely many possible outputs.
回归

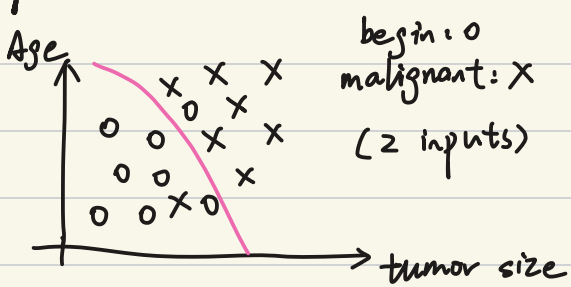
eg: Housing price prediction.



↳ Classification: Predict categories small number of possible outputs.
分类

eg: Cancer detection.





2. Unsupervised Learing Find sth. interesting in unlabled data.

↳ Clustering: Group similar data
聚类 eg: Google news. 一类新闻(均有关键词…)

↳ Anomaly detection: Find unusual data
异常检测

↳ Dimensionality reduction: Compress data.(压缩数据)

Made with Goodnotes

✱ Linear Regression Model.

① $f_{w,b}(x) = wx + b$. single feature

cost function $J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^i) - y^i)^2$

Gradient Descent 梯度下降 Algorithm

$w = w - \alpha \frac{d}{dw} J(w,b)$ α: (Learning Rate) $\in (0,1)$

$b = b - \alpha \frac{d}{db} J(w,b)$ Repeat Until Convergence.

Simultaneous update. 同时更新

$tmp_w = w - \alpha \frac{d}{dw} J(w,b)$

$tmp_b = b - \alpha \frac{d}{db} J(w,b)$

$w = tmp_w$; $b = tmp_b$

$\frac{d}{dw} J(w,b) = \frac{d}{dw} \frac{1}{2m}\sum_{i=1}^{m}(\underbrace{f_{w,b}(x^{(i)})}_{wx^{(i)}+b} - y^{(i)})^2$

$= \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^i) - y^i)x^i$

$\frac{d}{db} J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^i) - y^i)$

Made with Goodnotes

② Multiple features (variables) → [4 3 1 2]

x_j: j^{th} feature $\vec{x}^{(i)}$: features of ith example

$\therefore f_{w,b}(x) = w_1 x_1 + \cdots + w_n x_n + b$

$= \vec{w}\vec{x} + b$

$w_1 = w_1 - \alpha \frac{d}{dw_1} J(\vec{w}, b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_1^{(i)}$

$\vdots$

$w_n = w_n - \alpha \frac{d}{dw_n} J(\vec{w}, b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_n^{(i)}$

$b = b - \alpha \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$

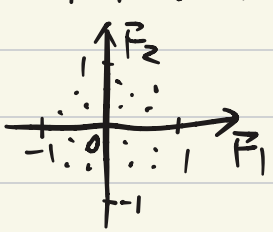
Feature scaling 特征缩放

① $\frac{x_1}{\max(x_1)}$



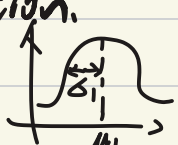
② Mean normalization.

$x_1' = \frac{x_1 - \mu_1}{\max - \min}$ → average



③ Z-score normalization.

$x_1 = \frac{x_1 - \mu_1}{\sigma_1}$



Made with Goodnotes

Feature Engineering:

Using intuition to design new features, by transforming or combining original features

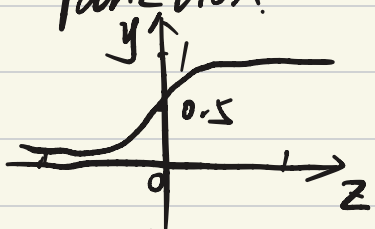
⇒ Polynomial Regression 多项式回归.

底层回归. 上层分类.

Logestic Regression (逻辑回归) Actually Classification.

method: sigmoid / logistic function.

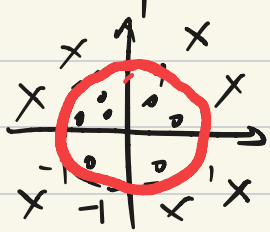
$$g(z) = \frac{1}{1+e^{-z}} \quad \in (0,1)$$



$$z = \vec{w} \cdot \vec{x} + b$$

Decision Boundary

eg:

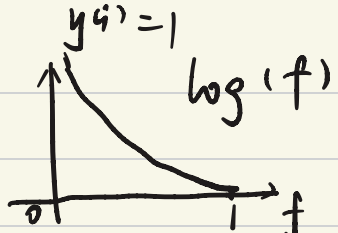


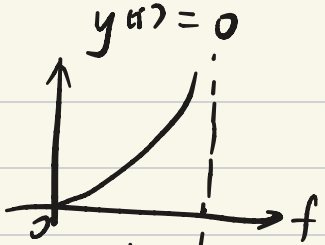
Cost function for logestic regression (Convex)

$$L\left(f_{\vec{w},b}(x^{(i)}), y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}(x^{(i)})\right) & \text{if } y^{(i)}=1 \\ -\log\left(1-f_{\vec{w},b}(x^{(i)})\right) & \text{if } y^{(i)}=0 \end{cases}$$

Made with Goodnotes

$f \in (0, 1)$





$f_{\vec{w},b}(x^{(i)})$ 是输出，是逻辑回归的预测。$y^{(i)}$ 则是实际。

比如 $y^{(i)}=1$，实际是1。则 $f(x^{(i)})$ 越接近1，Loss更小。

∴ Cost function for logestic regression:

$$J(\vec{w},b)=\frac{1}{m}\sum_{i=1}^{m} L\left(f_{\vec{w},b}(x^{(i)}), y^{(i)}\right)$$

$$\rightarrow \begin{cases} -\log[f(x)] & \text{if } y^{(i)}=1 \\ -\log[1-f(x)] & \text{if } y^{(i)}=0 \end{cases}$$

☆ Simplified Cost Function for Logestic Regression.

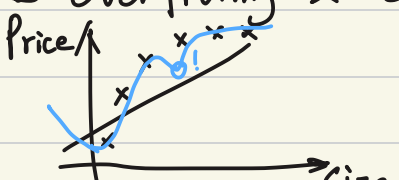
(其实就是合并起来) $J(\vec{w},b)=\frac{1}{m}\sum_{i=1}^{m} L\left(f(x^{(i)}), y^{(i)}\right)$

∴ $L\left(f(x^{(i)}), y^{(i)}\right) = -y^{(i)}\log[f(x^{(i)})] - (1-y^{(i)})\log[1-f(x^{(i)})]$

☆ Gradient Descent on Logestic Regression

$$\begin{cases} w_j = w_j - \alpha \cdot \frac{d}{dw_j} J(\vec{w},b) \\ b = b - \alpha \cdot \frac{d}{db} \cdot J(\vec{w},b) \end{cases}$$

☆ Overfitting ☆ Underfitting Problems for Regression.



- underfit ⇒ high bias $(w,x+b)$ (Don't fit)
- overfit ⇒ high variance 方差 (fit the data well)

Made with Goodnotes

✡ Addressing overfitting.

1. More data. 2. Select Features

3. Reduce size of parameters. ($w_i \downarrow$) "Regularization".

✡ "Regularization." (Some changes on cost function).

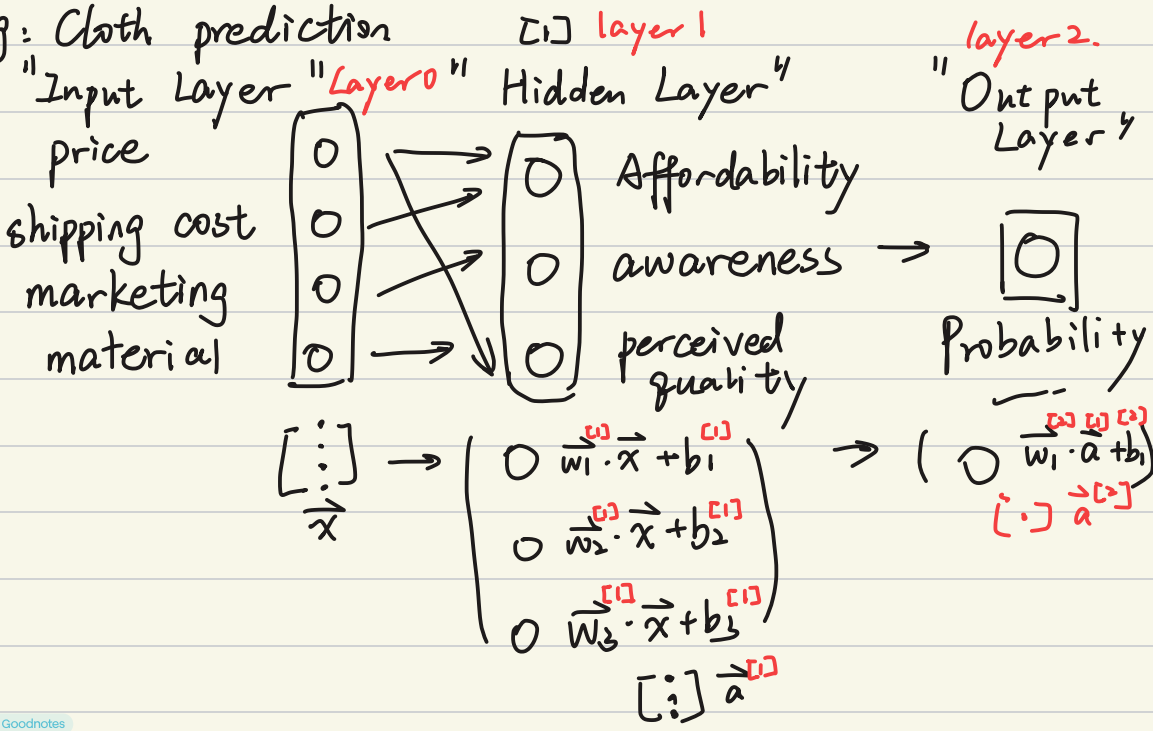
$$J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m} L(f(x^{(i)}), y^{(i)}) + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

为了减小 W_j，避免 overfit

Deep Learning.

✡ What is neural network.

eg: Cloth prediction

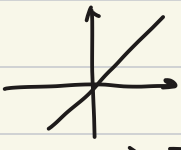
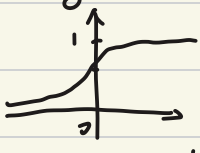


Made with Goodnotes

Train a Neural Network in TensorFlow (code)

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
    model = Sequential([
        Dense(units=25, activation='sigmoid'),
        Dense(units=15, activation='sigmoid'),
        Dense(units=1, activation='sigmoid'),
                ])
from tensorflow.keras.losses import BinaryCrossentropy
    model.compile(loss=BinaryCrossentropy())
    model.fit(X, Y, epochs=100)
```

总结：激活函数

Linear	Sigmoid	ReLU
		
$g(z)=\vec{w}\cdot\vec{x}+b$	$g(z)=\frac{1}{1+e^{-z}}$	$g(z)=\max(0,z)$
通用	Regression	Binary Classification	Regression
例子	$y=+/-$	(输出层)	$y=+$
(Don't use in hidden layer)			(推荐为隐藏层)

Made with Goodnotes

Multi-class classification (Softmax Regression)

N possible outputs.

$z_j = \vec{w_j} \cdot \vec{x} + b_j \quad j = 1, 2, \cdots, n$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^{n} e^{z_k}}$$

$$Loss = \begin{cases} -\log a_1 & \text{if } y=1 \\ \vdots & \\ -\log a_{10} & \text{if } y=10 \end{cases}$$

* code: activation = 'softmax' ; loss = SparseCategoricalCrossEntropy

More accurate:

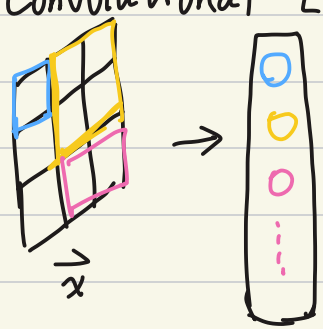
model.compile(loss = SCCE(from_logits = True))

Adam Algorithm (change α) faster than Gradient Descent.

↳ Adaptive Moment estimation

Code: model.compile(optimizer = tf.keras.optimizer.Adam(learning_rate = 1e-3), loss = ...)

Convolutional Layer (卷积层) 神经元仅接收输入部分.



Advantage: ① faster

② Need less training data.

Made with Goodnotes

Evaluating a model. (70% traing set — 30% test set)

1. Fit parameters take Liner Regression for example

$$J(\vec{w},b)=\left[\frac{1}{2m_{train}}\sum_{i=1}^{m_{train}}(f_{\vec{w},b}(x)-y)^2+\frac{\lambda}{2m_{train}}\sum w_j^2\right]$$

2. Compute test error

$$J_{test}(\vec{w},b)=\frac{1}{2m_{test}}\left[\sum(f(x)-y)^2\right]$$

依旧可能 overfit

3. Compute train error.

$J_{train}(\cdots)=\cdots$ (no $\frac{\lambda}{2m}\sum w_j^2$)

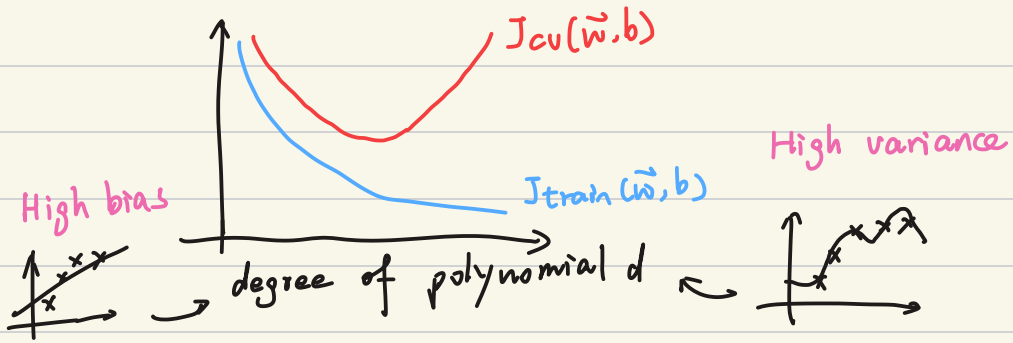
↓ optimize

60% training set — 20% cross validation 20% test

↳ 模拟 ↳ 最终考核 set

被多 w,b. 只选择. (只能用一次)

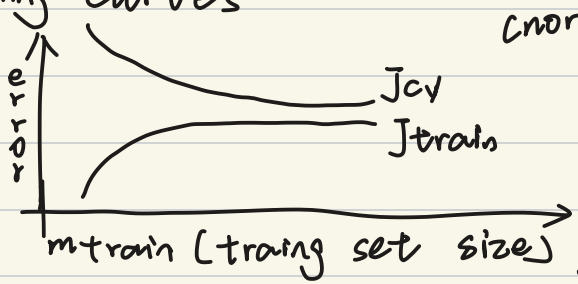
bias and variance



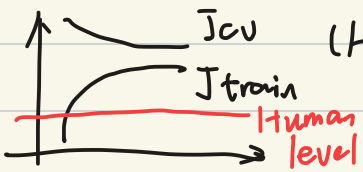
establish a baseline (Human performance)

Made with Goodnotes

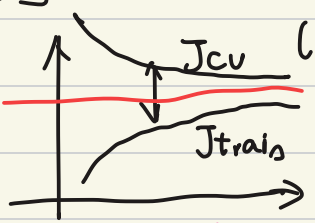
※ Learning curves



(normal)



(High bias)



(High variance)

不可增加数据提升效果

可增加数据提升效果

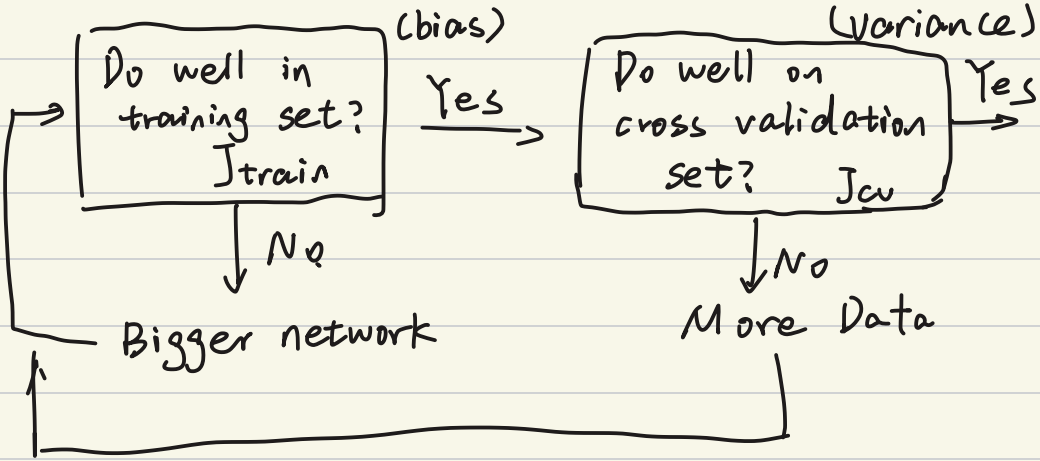
$$J(\vec{w},b)=\frac{1}{2m}\sum_{i=1}^{m}(f(x)-y)^2+\frac{\lambda}{2m}\sum_{j=1}^{n}w_j^2$$

总结: If unacceptable error in prediction.

- Fix high bias:
  - → Try getting additional features
  - → Adding polynomial features
  - → Decreasing λ
- Fix high variance:
  - → More training examples
  - → Less features
  - → Increasing λ

Made with Goodnotes

Neural networks solve bias and variance:



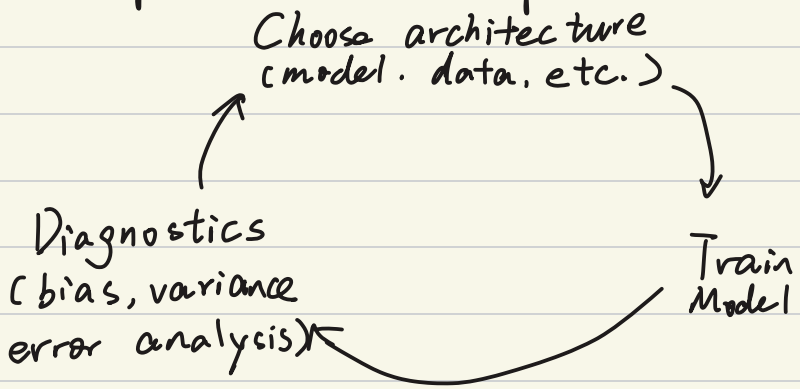
Neural network regularization (code)

```
layer_1 = Dense (units =25, activation = "relu", kernel_regularizer=L2(0.01))
layer_2 = ..
   :
model = Sequential ([layer_1, layer_2, ---])
```

- 只要正则化，更大的神经网络不会有坏处（只会下降速度）
- Larger neural networks are low bias machines

Made with Goodnotes

• Iterative loop of ML development



☆ Adding data:

• add based on error analysis (选出错最高)

• data augmentation (数据增强) 人为通过一个数据变换成更多数据:

eg: add background noise (语音识别)

• Data syntnesis (数据合成)

* skewed data sets

eg. rare disease diagnose (人群中仅0.5%患病. 因此直接预测 $y=0$ 可达 99.5% 准确率. 但实际不准确)

* Precision / recall

Predicted Class \ Actual Class	1	0
1	True Positive 15	False Positive 5
0	False Negative 10	True Negative 70
	25	75

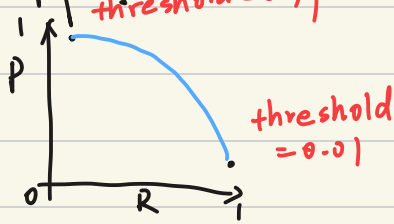
Precision: $\frac{15}{15+5} = 0.75$ — of all that we predicted 1, what fraction actually have disease

Recall $\frac{15}{15+10} = 0.6$ — of all that have disease, what fraction did we correctly predict

* Trading off precision & recall

提高阈值 (一般0.5) : precision ↑ recall ↓

降低 : precision ↓ recall ↑

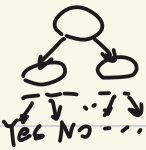


→ F_1 score : $= \frac{1}{\frac{1}{2}(\frac{1}{P}+\frac{1}{R})} = 2\frac{PR}{R+P}$

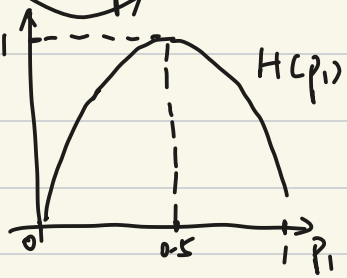
Harmonic mean 调和平均数 ✓

Made with Goodnotes

Decision Tree.



★ Entropy (熵) — a measure of impurity



p_1: fraction of cats

$p_0 = 1 - p_1$

$$H(p_1) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

★ Choosing a split (eg: discriminate cats & dogs)

$p_0 = \frac{5}{10}$ ← Ear shape

$H(p_0) = 1$

原本有5猫 5狗 . . . . .

$p_1 = \frac{4}{5}$ $p_2 = \frac{1}{5}$

$H(p_1) = 0.72$ $H(p_2) = 0.72$

⇒ 加权 $\left[\frac{5}{10} H(p_1) + \frac{5}{10} H(p_2)\right]$

$\frac{5}{10}$: 左一共数 / 总共

$H(p_0)$ − ⇒ Information gain (entropy decrease) 熵减少量

一般式: $H(p_1^{root}) - (w^{left} H(p_1^{left}) + w^{right} H(p_1^{right}))$

Decision Tree Learning

- Start with all examples at the root node
- Calculate information gain for all possible features, and pick the one with the highest information gain
- Split dataset according to selected feature, and create left and right branches of the tree
- Keep repeating splitting process until stopping criteria is met:
  - When a node is 100% one class
  - When splitting a node will result in the tree exceeding a maximum depth
  - Information gain from additional splits is less than threshold
  - When number of examples in a node is below a threshold

Made with Goodnotes

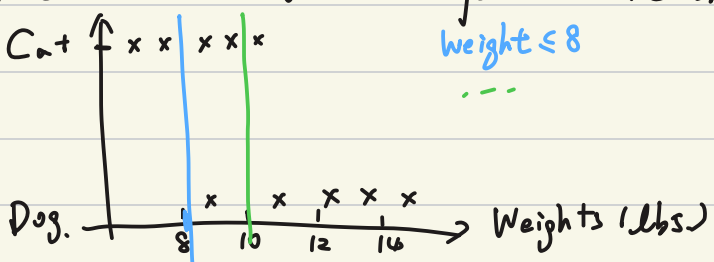
✲ One-hot encoding (独热编码)

eg: Features with many possible features (离散型)

	Ear Shape	Pointy	Oval	Floppy
A	Pointy	1	0	0
B	Oval	0	1	0
C	Floppy	0	0	1

⇒ 变成了每列只有两个取值.

✲ Continuous valued features (连续型)



Weight ≤ 8

...

$H(0.5) - [\frac{2}{10} H(\frac{2}{2}) + \frac{8}{10} H(\frac{3}{8})]$ ...

✲ Regression Tree

eg: Using ear, face shape, Whiskers to predict weight

→ Choosing a split (Variance)

Variance at root node $- (w^{left} \cdot v^{left} + w^{right} \cdot v^{right})$

选最大的, 让方差下降最大.

Made with Goodnotes

Made with Goodnotes